Компьютерные науки и анализ данных

Yandex

Faculty of Computer science
Higher School of Economics

Программирование на Python

лектор: Сергей Горшков

План

- Абстрактные классы
- Слоты
- Работа с файловой системой
- Работа с аргументами командной строки
- Пакеты и модули. Файл __init__.py, использование __all__
- Менеджер пакетов pip, virtualenv
- the walrus operator (`:=`) и еще пара нововведение Python 3.8

Абстрактные классы

- Иногда нам нужно лишь задать набор методов, запрещая создавать экземпляр класса
- Следуя принципу полиморфизма можем общаться со всеми потомками, используя задаваемый интерфейс
- Методы, реализация которых возлагается на классы-потомки также абстрактные (в родительском классе они не реализованы)
- Нужен механизм, позволяющий отслеживать наличие реализации

Абстрактные классы в Python

- from abc import ABC, abstractmethod
- @abstractmethod # декоратор
  do_smth(): pass
- Только декоратора недостаточно! Будет создаваться объект и даже вызываться помеченный декоратором метод:(
- Надо отнаследовать класс от абстрактного базового класса ABC
- Вот теперь всё работает как ожидается (при создании экземпляра вылетит ошибка типа)
- Без переопределения абстрактного метода в классе-наследнике его экземпляр также не будет создан

Слоты

- В питоне система управления атрибутами динамическая – они хранятся в __dict__. У каждого *экземпляра* класса есть этот словарь
- За это платим понижением скорости доступа к атрибутам и дополнительным расходом памяти.
- Есть много случаев, когда мы заранее знаем, какие атрибуты будут у класса
- Специальный атрибут __slots__ позволяет вам явно указать, какие атрибуты экземпляра вы ожидаете от экземпляров вашего объекта
- Теперь не можем добавлять в экземпляры случайные атрибуты
- Если явно указать значение __slots__, то словарь __dict__ создаваться не будет
- Колаб

Слоты. Наследование

- Значение __slots__ наследуется, однако это *не* предотвращает создание __dict__
- Таким образом, дочерние классы будут позволять добавлять динамические атрибуты, и добавляться они будут в __dict__, со всеми вытекающими расходами.
- Если нам нужно, чтобы и дочерний класс тоже был ограничен слотами, там придётся и в нём присвоить значение атрибуту __slots__. Кстати, дублировать уже указанные в родительском классе слоты не нужно.
- Множественное наследование, в котором в нескольких базовых классах определены слоты, обречено на ошибку типа
- Вот здесь можно почитать про метаклассы

Модуль os

- os.chdir(path) – смена директории
- os.getcwd() – текущая рабочая директория
- os.listdir(path=".") – список файлов и директорий в папке
- os.mkdir(path, mode=0o777, *, dir_fd=None) – создаёт директорию
- os.makedirs(path, mode=0o777, exist_ok=False) + промежуточные директории
- os.walk(top, topdown=True, onerror=None, followlinks=False) – генерация имён файлов в дереве каталогов, сверху вниз (topdown == True). Для каждого каталога функция walk возвращает кортеж (путь к каталогу, список каталогов, список файлов).
- os.path

Работа с аргументами командной строки

- sys.argv - список аргументов командной строки, передаваемых сценарию Python. sys.argv[0] является именем скрипта (пустой строкой в интерактивной оболочке)
- Модуль argparse
- ```
  import argparse
  parser = argparse.ArgumentParser()
  parser.add_argument('-1', '--one' , help='option One')
  parser.parse_args()
  ```
- https://rtfm.co.ua/python-modul-argparse-opcii-komandnoj-stroki-v-primerax/#add_argument_-_name_%D0%B8%D0%BB%D0%B8_flags
- https://docs.python.org/3/library/argparse.html
  ```

Модули

- Модуль – любой .py файл; посмотреть что внутри – import module; dir(module)
- Пути поиска модулей указаны в sys.path (текущая директория + директории, в которых установлен Python). Либо же переменная окружения PYTHONPATH
- sys.executable – путь к используемому интерпретатору
- Подключение модуля – через import
- При импортировании модуля его код исполняется полностью!
    - import this
- Можно проверять, запущен ли скрипт как программа или импортирован: __name__
- Она равна __main__, если скрипт запущен в качестве главной программы, или же имя, если он импортирован

.pyc-файлы и инициализация модулей

- При импорте модуля появляется файл .pyc в папке __pycache__
- Это скомпилированный файл Python. Python компилирует файлы в байт-код Python, так что ему не придется анализировать файлы при каждой загрузке модулей.
- Если файл .pyc для модуля существует, то загружается именно он
- При первой загрузке модуля в работающий скрипт он инициализируется однократным выполнением кода в модуле. Если другой модуль в коде импортирует тот же модуль, то он будет загружен не дважды, а один раз

- Запуск модуля в терминале: python -m module_name
  - -m mod : run library module as a script (terminates option list)

Пакеты

- Пакет – пространство имён, содержащее модули и другие пакеты
- Это каталоги, в которых определен специальный файл __init__.py (<Python3.3 он нужен обязательно, после – опционально)
- Этот файл может быть пустым
- Его наличие говорит о том, что каталог, который его содержит, является пакетом и его можно импортировать точно так же как модуль
- Файл __init__.py может решить, какие модули пакет экспортирует, сохраняя при этом другие модули внутренними, переопределив переменную __all__
- Внутренний импорт имени в __init__.py: `from path.to.name import name`
- Пожалуй, исчерпывающее трехчасовое видео на английском – https://youtu.be/0oTh1CXRaQ0 про модули и пакеты (всё немного сложнее)

School of Economics
Computer science

Менеджер пакетов pip

- pip help - помощь по доступным командам.
- pip install package_name - установка пакета(ов).
- pip uninstall package_name - удаление пакета(ов).
- pip list - список установленных пакетов.
- pip show package_name - показывает информацию об установленном пакете.
- pip search - поиск пакетов по имени.
- pip install -U - обновление пакета(ов).
- Можно использовать и локальный pip вашей компании
- pip freeze > requirements.txt
- Обратите внимание, что если установлены разные версии python, то лучше использовать pip3, pip3.6, и т.п.

Виртуальное окружение

- Модуль venv
- Python используется в скриптах в ОС
- python3 -m venv venv_name (venv_name – имя папки, куда надо сохранить зависимости проекта. Часто называют просто venv)
- which python3
- Если виртуальное окружение уже есть:
  - Активировать его: source venv_name/bin/activate
- Выйти из окружения: deactivate
- Не ставит пакеты при конфликте зависимостей pip>20.2

Оператор "морж" (>= 3.8)

- Определяет переменную в границах выражения в контексте
- Запись $x := y$ – отношение ‘x определяется как y’.
- Присвоение имени позволяет использовать «описательное» имя вместо более длинного выражения, а также упрощает повторное использование значений.
- В настоящее время это возможно сделать лишь в виде инструкции, что делает эту операцию недоступной при генерации списков
- `a := 3 # должно быть a=3.`
- `a = b := 4 # должно быть a=b=4`
- `a = (b := 4) # допустимо, но не рекомендуется`

Оператор "морж"

- `b:=4` не только устанавливает b равным четырём, но и возвращает его значение.
- Со скобками вокруг первого оператора (a := 3) он выполняется корректно, потому что использование скобок помечает всё внутри них как выражение. Операторы “морж” могут использоваться только внутри других выражений.
- `group = re.match(data).group(1) if re.match(data) else None`
-
```
match = re.match(data)
group = match.group(1) if match else None
```
- `group = match.group(1) if (match := re.match(data)) else None`

Позиционные аргументы и = в f-строках

- Всё, что находится перед / — строго позиционные аргументы, а все, что после * — только ключевые слова.
- `def f(a, b, /, c, d, *, e, f):`
  `print(a, b, c, d, e, f)`
- `f(10, 20, 30, d=40, e=50, f=60)` - valid
- `f(10, b=20, c=30, d=40, e=50, f=60)`- b cannot be a keyword argument
- `f(10, 20, 30, 40, 50, f=60)` - e must be a keyword argument

- `f"{chr(65) = }"` – если ставим = после имени переменной или выражения, то выведется значение переменной. Удобно для дебага

Ещё нововведения

- `c = a | b` и `a |= b` – слияние словарей (3.9)
- `.removeprefix` и `.removesuffix` для строк (3.9)
- `list(zip(range(3), ['fee', 'fi', 'fo', 'fum'], strict=True))` – добавлено ключевое слово strict, при указании которого осуществляется проверка на одинаковую длину перебираемых аргументов. (3.10)
- `FileName: TypeAlias = str` – создание псевдонимов для типов (3.10)
- `def parse(file: FileName) -> None: pass`

Match-case (3.10)

```
def func(day):
    match day:
        case "Monday":
            return "Here we go again..."
        case "Friday":
            return "Happy Friday!"
        case "Saturday" | "Sunday":  # Multiple literals can be
combined with `|`
            return "Yay, weekend!"
        case _:
            return "Just another day..."
```

Match-case (3.10)

```
def func(person):  # person = (name, age, gender)
    match person:
        case (name, _, "male"):
            print(f"{name} is man.")
        case (name, _, "female"):
            print(f"{name} is woman.")
        case (name, age, gender):
            print(f"{name} is {age} old.")


func(("John", 25, "male"))
# John is man.
```

Faculty of Computer Science
Higher School of Economics

sgorshkov@hse.ru
tg: @Seragor98